

# TypeScript & Módulos para o Filo CRM

---

Do zero ao primeiro módulo publicado: instalação, configuração, TypeScript e extensões Filo.

---

## **TypeScript & Módulos para o Filo CRM**

Guia do desenvolvedor — 1ª edição, 2026

Autor: Equipe W3TI SERVIÇOS DE INFORMÁTICA LTDA

© 2026 W3TI SERVIÇOS DE INFORMÁTICA LTDA. Todos os direitos reservados.

Filo CRM® é marca registrada de W3TI SERVIÇOS DE INFORMÁTICA LTDA.

TypeScript® é marca registrada de Microsoft Corporation.

Node.js® é marca registrada de OpenJS Foundation.

Visual Studio Code® é marca registrada de Microsoft Corporation.

Este documento é distribuído exclusivamente a licenciados do Filo CRM.

É vedada a reprodução total ou parcial sem autorização expressa da W3TI.

# Sumário

---

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Prefácio .....</b>                                                | <b>5</b>  |
| <b>Capítulo 01 — Introdução ao Ambiente de Desenvolvimento .....</b> | <b>6</b>  |
| 1.1 Por que TypeScript? .....                                        | 6         |
| 1.2 Instalando o Node.js .....                                       | 7         |
| 1.3 Instalando e configurando o VS Code .....                        | 9         |
| 1.4 Extensões recomendadas para TypeScript .....                     | 11        |
| <b>Capítulo 02 — Fundamentos de TypeScript .....</b>                 | <b>13</b> |
| 2.1 Criando e executando um projeto TypeScript .....                 | 13        |
| 2.2 Tipos primitivos e anotações .....                               | 16        |
| 2.3 Interfaces e tipos customizados .....                            | 19        |
| 2.4 Funções tipadas .....                                            | 22        |
| 2.5 Classes e encapsulamento .....                                   | 25        |
| 2.6 Genéricos .....                                                  | 28        |
| 2.7 Async/Await e Promises .....                                     | 30        |
| 2.8 Módulos ES — import e export .....                               | 33        |
| <b>Capítulo 03 — O Sistema de Módulos do Filo CRM .....</b>          | <b>36</b> |
| 3.1 O que é um módulo Filo .....                                     | 36        |
| 3.2 Arquitetura e ciclo de vida .....                                | 38        |
| 3.3 O manifesto module.json .....                                    | 40        |
| 3.4 A Filo SDK — visão geral das APIs .....                          | 42        |
| <b>Capítulo 04 — A Filo SDK em Detalhes .....</b>                    | <b>45</b> |
| 4.1 Data API — consultando e gravando dados .....                    | 45        |
| 4.2 UI API — registrando telas e componentes .....                   | 49        |
| 4.3 Events API — reagindo a eventos do sistema .....                 | 53        |
| 4.4 Utils API — utilitários e armazenamento .....                    | 56        |
| <b>Capítulo 05 — Criando o Primeiro Módulo .....</b>                 | <b>59</b> |
| 5.1 Scaffold com o Filo Studio .....                                 | 59        |
| 5.2 Estrutura de arquivos do módulo .....                            | 61        |
| 5.3 Implementando init() e destroy() .....                           | 63        |
| 5.4 Registrando um item na sidebar .....                             | 65        |
| 5.5 Compilando e testando com hot reload .....                       | 67        |
| <b>Capítulo 06 — Módulo Completo: Histórico de Interações .....</b>  | <b>69</b> |
| 6.1 Planejamento e manifesto .....                                   | 69        |
| 6.2 Criando tabela própria do módulo .....                           | 71        |
| 6.3 Tela de listagem com busca .....                                 | 73        |
| 6.4 Formulário de nova interação .....                               | 76        |
| 6.5 Publicando o módulo .....                                        | 78        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Capítulo 07 — Boas Práticas e Distribuição .....</b> | <b>80</b> |
| 7.1 Segurança e permissões mínimas .....                | 80        |
| 7.2 Tratamento de erros .....                           | 82        |
| 7.3 Testes com node:test .....                          | 84        |
| 7.4 Versionamento e changelog .....                     | 86        |
| <b>Apêndice A — Referência da Filo SDK .....</b>        | <b>88</b> |
| <b>Apêndice B — Eventos nativos do Filo CRM .....</b>   | <b>90</b> |
| <b>Apêndice C — Referência do module.json .....</b>     | <b>92</b> |

# 00

## Prefácio

Este guia foi escrito para o desenvolvedor que deseja criar extensões para o **Filo CRM** — o sistema de gestão de relacionamentos com clientes da W3TI SERVIÇOS DE INFORMÁTICA LTDA. O livro parte do absoluto zero: instalação do ambiente de desenvolvimento, configuração do editor e primeiros passos com TypeScript.

A segunda metade do livro é dedicada inteiramente ao sistema de módulos do Filo. Você aprenderá a estrutura de um módulo, as APIs disponíveis para interagir com os dados e a interface do sistema, e construirá um módulo funcional do início ao fim.

■ **Pré-requisito:** conhecimento básico de lógica de programação — variáveis, condicionais, laços e funções. Não é necessária experiência prévia com TypeScript, Node.js ou JavaScript.

Todos os exemplos de código estão disponíveis no repositório oficial em [github.com/w3ti/filo-modulos-exemplos](https://github.com/w3ti/filo-modulos-exemplos). Recomenda-se executar cada exemplo à medida que avança nos capítulos.

**Equipe W3TI — 2026**

# 01

## Introdução ao Ambiente de Desenvolvimento

*Por que TypeScript, como instalar o Node.js e configurar o VS Code.*

### 1.1 Por que TypeScript?

O Filo CRM é construído em **TypeScript**, um superconjunto tipado do JavaScript desenvolvido pela Microsoft. Para desenvolver módulos que se integram ao Filo, é necessário escrever código TypeScript.

A principal vantagem do TypeScript em relação ao JavaScript puro é a detecção de erros em tempo de desenvolvimento — antes mesmo de executar o programa. O compilador analisa o código e aponta inconsistências de tipos, propriedades inexistentes e chamadas incorretas de funções.

| Característica          | JavaScript           | TypeScript                  |
|-------------------------|----------------------|-----------------------------|
| Detecção de erros       | Em tempo de execução | Em tempo de desenvolvimento |
| Autocompletar no editor | Limitado             | Completo (IntelliSense)     |
| Documentação inline     | Manual               | Gerada pelos tipos          |
| Refatoração segura      | Arriscada            | Assistida pelo compilador   |
| Curva de aprendizado    | Menor                | Progressiva                 |

### 1.2 Instalando o Node.js

O TypeScript é executado sobre o **Node.js**, um ambiente de execução JavaScript multiplataforma. O Node.js inclui o gerenciador de pacotes **npm**, utilizado para instalar o compilador TypeScript e as dependências dos módulos.

#### Windows

- 1 Acesse [nodejs.org](https://nodejs.org) e clique em **LTS** para baixar a versão estável recomendada.

•

- 2 Execute o instalador baixado (node-v20.x.x-x64.msi) e siga o assistente com as opções padrão.
- 3 Ao final, marque a opção **"Automatically install the necessary tools"** se aparecer.
- 4 Abra o **Prompt de Comando** ou o **PowerShell** e verifique a instalação:

Verificando a instalação no Windows

```
1 node --version # Ex: v20.14.0
2 npm --version # Ex: 10.7.0
```

## macOS

No macOS, a forma recomendada é instalar via **Homebrew**:

Instalando Node.js no macOS via Homebrew

```
1 # Instalar o Homebrew (caso não tenha)
2 # Instalar o Homebrew
3 # Acesse: raw.githubusercontent.com/Homebrew/install/HEAD/install.sh
4 /bin/bash -c "$(curl -fsSL [URL acima])"
5
6 # Instalar o Node.js
7 brew install node
8
9 # Verificar
10 node --version
11 npm --version
```

## Linux (Ubuntu/Debian)

Instalando Node.js no Ubuntu/Debian

```
1 # Adicionar repositório NodeSource
2 curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
3
4 # Instalar
5 sudo apt-get install -y nodejs
6
7 # Verificar
8 node --version
9 npm --version
```

- Utilize sempre a versão **LTS (Long-Term Support)** do Node.js. Versões LTS são mantidas por pelo menos 30 meses e são as mais estáveis para desenvolvimento.

## 1.3 Instalando e configurando o VS Code

O **Visual Studio Code** (VS Code) é o editor recomendado para o desenvolvimento de módulos Filo. Ele oferece suporte nativo a TypeScript com IntelliSense, depuração integrada e um ecossistema rico de extensões.

- 1 Acesse [code.visualstudio.com](https://code.visualstudio.com) e baixe a versão para seu sistema operacional.
  -
- 2 Execute o instalador. No Windows, marque **"Add to PATH"** durante a instalação.
  -
- 3 Após instalar, verifique se o comando `code` está disponível no terminal:
  -

Verificando o VS Code no terminal

```
1 code --version # Ex: 1.89.0
```

Com o VS Code instalado, algumas configurações globais melhoram a experiência com TypeScript. Acesse **File** → **Preferences** → **Settings** (ou **Ctrl+,**) e configure:

settings.json — configurações recomendadas

```
1 {
2   "editor.formatOnSave": true,
3   "editor.defaultFormatter": "esbenp.prettier-vscode",
4   "editor.tabSize": 2,
5   "editor.insertSpaces": true,
6   "typescript.preferences.quoteStyle": "single",
7   "typescript.updateImportsOnFileMove.enabled": "always",
8   "files.autoSave": "onWindowChange"
9 }
```

## 1.4 Extensões recomendadas para TypeScript

Instale as seguintes extensões pelo painel de extensões do VS Code (**Ctrl+Shift+X**):

| Extensão | ID                     | Finalidade                       |
|----------|------------------------|----------------------------------|
| ESLint   | dbaeumer.vscode-eslint | Análise estática e boas práticas |



| Extensão          | ID                                 | Finalidade                           |
|-------------------|------------------------------------|--------------------------------------|
| Prettier          | esbenp.prettier-vscode             | Formatação automática de código      |
| Error Lens        | usernamehw.errorlens               | Exibe erros inline no código         |
| TypeScript Hero   | rbbit.typescript-hero              | Organiza importações automaticamente |
| Path Intellisense | christian-kohler.path-intellisense | Autocompletar de caminhos de arquivo |
| GitLens           | eamodio.gitlens                    | Histórico Git integrado ao editor    |

# 02

## Fundamentos de TypeScript

*Tipos, interfaces, classes, async/await e módulos ES.*

### 2.1 Criando e executando um projeto TypeScript

Antes de escrever código TypeScript, é necessário criar um projeto com a estrutura correta. Abra o terminal, crie uma pasta e inicialize o projeto:

Terminal — inicializando um projeto TypeScript

```
1  # Criar pasta e entrar nela
2  mkdir meu-projeto && cd meu-projeto
3
4  # Inicializar o projeto Node.js
5  npm init -y
6
7  # Instalar o TypeScript como dependência de desenvolvimento
8  npm install --save-dev typescript
9
10 # Criar o arquivo de configuração do TypeScript
11 npx tsc --init
```

O comando `npx tsc --init` cria o arquivo `tsconfig.json` na raiz do projeto. Abra-o no VS Code e substitua o conteúdo pelo seguinte:

tsconfig.json — configuração base

```
1  {
2    "compilerOptions": {
3      "target": "ES2022",
4      "module": "CommonJS",
5      "lib": ["ES2022", "DOM"],
6      "outDir": "./dist",
7      "rootDir": "./src",
8      "strict": true,
9      "esModuleInterop": true,
10     "skipLibCheck": true,
11     "forceConsistentCasingInFileNames": true,
```

```
12  "resolveJsonModule": true
13  },
14  "include": ["src/**/*"],
15  "exclude": ["node_modules", "dist"]
16  }
```

Crie a estrutura de pastas e o primeiro arquivo TypeScript:

Criando a estrutura inicial

```
1  # Criar pasta src
2  mkdir src
3
4  # Criar o primeiro arquivo
5  echo "console.log('Olá, TypeScript!');" > src/index.ts
```

Para compilar e executar:

Compilando e executando

```
1  # Compilar TypeScript → JavaScript
2  npx tsc
3
4  # Executar o JavaScript gerado
5  node dist/index.js
6
7  # Saída esperada:
8  # Olá, TypeScript!
```

Para agilizar o desenvolvimento, instale o `ts-node` — que executa TypeScript diretamente sem precisar compilar manualmente:

Executando TypeScript diretamente com `ts-node`

```
1  npm install --save-dev ts-node
2
3  # Executar diretamente
4  npx ts-node src/index.ts
5
6  # Saída:
7  # Olá, TypeScript!
```

- Adicione scripts ao `package.json` para facilitar o uso: `"dev": "ts-node src/index.ts"` e `"build": "tsc"`. Assim, basta executar `npm run dev` para rodar o projeto.

## 2.2 Tipos primitivos e anotações

A principal diferença entre TypeScript e JavaScript é a adição de **anotações de tipo**. Elas informam ao compilador qual tipo de valor uma variável, parâmetro ou retorno de função deve conter.

Tipos primitivos, inferência e union types

```
1 // Tipos primitivos
2 let nome: string = "Carlos Mendes";
3 let idade: number = 42;
4 let ativo: boolean = true;
5
6 // Inferência automática de tipo
7 // (TypeScript detecta o tipo pelo valor inicial)
8 let empresa = "Tecnovar Ltda"; // string
9 let preco = 1500.00; // number
10 let publicado = false; // boolean
11
12 // Arrays tipados
13 let nomes: string[] = ["Ana", "Carlos", "Pedro"];
14 let valores: number[] = [100, 200, 300];
15
16 // Union types – pode ser um tipo OU outro
17 let codigo: string | null = null;
18 let contato: string | number = "telefone";
19 contato = 11999990000; // também válido
20
21 // Operador ?? (coalescência nula)
22 let apelido: string | null = null;
23 let exibir = apelido ?? "Sem apelido"; // "Sem apelido"
```

## 2.3 Interfaces e tipos customizados

Uma **interface** define a estrutura de um objeto — quais propriedades ele deve ter e de que tipo cada uma é. É o mecanismo central de tipagem utilizado em toda a Filo SDK.

Interfaces e alias de tipo

```
1 // Definindo uma interface
2 interface Contato {
3   id: string;
4   nome: string;
5   email: string | null;
6   empresa?: string; // ? = propriedade opcional
```

```
7   criadoEm: Date;
8   }
9
10  // Criando um objeto que satisfaz a interface
11  const contato: Contato = {
12    id: "c-001",
13    nome: "Ana Lima",
14    email: "ana@inova.com.br",
15    criadoEm: new Date(),
16  };
17
18  // TypeScript protege contra erros comuns
19  // contato.telefone = "11999"; ← ERRO: propriedade não existe
20  // contato.nome = 42; ← ERRO: esperava string, recebeu number
21
22  // Alias de tipo – útil para literais e uniões
23  type StatusOportunidade =
24    | "prospeccao"
25    | "qualificacao"
26    | "proposta"
27    | "fechado_ganho"
28    | "fechado_perdido";
29
30  let status: StatusOportunidade = "proposta"; // ✓
31  // status = "em_aberto"; ← ERRO: valor não permitido
```

## 2.4 Funções tipadas

Em TypeScript, tanto os parâmetros quanto o retorno de uma função podem — e devem — ter tipos declarados. Isso garante que a função seja chamada corretamente em todo o código.

Funções com tipos explícitos

```
1   // Parâmetros e retorno tipados
2   function calcularDesconto(preco: number, pct: number): number {
3     return preco * (1 - pct / 100);
4   }
5
6   calcularDesconto(1000, 10); // ✓ retorna 900
7   // calcularDesconto("mil", 10); ← ERRO
8
9   // Arrow function tipada
```

```
10 const formatarBRL = (valor: number): string => {
11   return valor.toLocaleString("pt-BR", {
12     style: "currency",
13     currency: "BRL",
14   });
15 };
16
17 formatarBRL(1234.56); // "R$ 1.234,56"
18
19 // Parâmetro opcional e valor padrão
20 function saudar(nome: string, titulo: string = "Sr."): string {
21   return `Olá, ${titulo} ${nome}!`;
22 }
23
24 saudar("Mendes"); // "Olá, Sr. Mendes!"
25 saudar("Lima", "Dra."); // "Olá, Dra. Lima!"
26
27 // void – função que não retorna valor
28 function exibirErro(mensagem: string): void {
29   console.error(`[ERRO] ${mensagem}`);
30 }
```

## 2.5 Classes e encapsulamento

Classes permitem organizar código e dados relacionados em uma única estrutura. TypeScript adiciona os modificadores `public`, `private` e `protected` para controlar o acesso às propriedades e métodos.

Classe genérica com modificadores de acesso

```
1  class Repositorio<T extends { id: string }> {
2    private itens: T[] = [];
3
4    public adicionar(item: T): void {
5      this.itens.push(item);
6    }
7
8    public buscarPorId(id: string): T | undefined {
9      return this.itens.find((i) => i.id === id);
10   }
11
12   public listar(): T[] {
13     return [...this.itens]; // retorna cópia
```

```
14  }
15
16  public get total(): number {
17    return this.itens.length;
18  }
19  }
20
21  // Usando a classe
22  const repo = new Repositorio<Contato>();
23  repo.adicionar({ id: "c-001", nome: "Ana", email: null, criadoEm: new Date() });
24  console.log(repo.total); // 1
25
26  // repo.itens; ← ERRO: propriedade privada
```

## 2.6 Genéricos

Genéricos permitem escrever código reutilizável que funciona com qualquer tipo, mantendo a segurança de tipos. A Filo SDK utiliza genéricos na Data API para retornar resultados fortemente tipados.

Genéricos em funções e interfaces

```
1  // Função genérica – T é definido na chamada
2  function primeiro<T>(array: T[]): T | undefined {
3    return array[0];
4  }
5
6  const p1 = primeiro(["Ana", "Carlos"]); // string | undefined
7  const p2 = primeiro([10, 20, 30]); // number | undefined
8
9  // Interface genérica para resultados paginados
10 interface Pagina<T> {
11   dados: T[];
12   total: number;
13   pagina: number;
14   porPagina: number;
15 }
16
17 // Uso com a Data API do Filo
18 const pagina: Pagina<Contato> = {
19   dados: /* lista de contatos */,
20   total: 148,
21   pagina: 1,
```

```
22   porPagina: 20,  
23   };
```

## 2.7 Async/Await e Promises

Toda operação da Filo SDK que envolve acesso a dados, rede ou armazenamento é **assíncrona** — ela retorna uma **Promise**. O par `async/await` é a forma moderna de trabalhar com Promises.

Async/await com tratamento de erros

```
1  // Uma Promise representa um valor futuro  
2  // async marca a função como assíncrona  
3  async function buscarContato(id: string): Promise<Contato | null> {  
4    // await aguarda a Promise ser resolvida  
5    const resultado = await chamarAPI(`/contacts/${id}`);  
6    return resultado;  
7  }  
8  
9  // Tratamento de erros com try/catch  
10 async function salvar(contato: Contato): Promise<void> {  
11   try {  
12     await gravarNoBanco(contato);  
13     console.log("Contato salvo!");  
14   } catch (erro) {  
15     console.error("Falha ao salvar:", erro);  
16   }  
17 }  
18  
19 // Executar múltiplas Promises em paralelo  
20 async function carregarDashboard(): Promise<void> {  
21   const [contatos, oportunidades] = await Promise.all([  
22     buscarContatos(),  
23     buscarOportunidades(),  
24   ]);  
25   renderizar(contatos, oportunidades);  
26 }
```

- Sempre use `try/catch` em chamadas assíncronas. Um erro não tratado em um módulo Filo pode impactar a estabilidade do sistema para o usuário.

## 2.8 Módulos ES — import e export





# 03

## O Sistema de Módulos do Filo CRM

*Arquitetura, ciclo de vida e o arquivo de manifesto.*

### 3.1 O que é um módulo Filo

O Filo CRM foi projetado para ser extensível. Funcionalidades específicas de um segmento ou empresa podem ser adicionadas por meio de **módulos** — pacotes TypeScript que se integram à interface e aos dados do sistema sem modificar o código principal do Filo.

Um módulo pode:

- Adicionar itens à barra lateral de navegação
- Registrar abas extras nas telas de contatos e oportunidades
- Exibir widgets no dashboard
- Reagir a eventos do sistema (oportunidade fechada, contato criado etc.)
- Criar e consultar suas próprias tabelas no banco de dados do Filo
- Exibir modais e notificações (toasts)

■ Módulos rodam em um ambiente isolado (*sandbox*). Eles não têm acesso direto ao sistema de arquivos, à rede ou ao processo principal do Filo. Toda interação ocorre exclusivamente pela **Filo SDK**.

### 3.2 Arquitetura e ciclo de vida

Quando o Filo inicia, o **Module Runtime** escaneia a pasta `~/filo-modulos/` em busca de módulos instalados. Para cada módulo ativo, ele:

- 1 Lê e valida o arquivo `module.json`.
- 
- 2 Carrega o JavaScript compilado em um contexto isolado.
- 
- 3 Injeta a instância da Filo SDK no contexto do módulo.
-

#### 4 Chama a função `init(sdk)` exportada pelo módulo.

▪

Quando o usuário desativa ou atualiza o módulo, o runtime chama `destroy()` para que o módulo limpe seus recursos (cancelar subscrições de eventos, limpar timers etc.).

Estrutura mínima de um módulo Filo

```
1  import type { FiloSDK } from "@w3ti/filo-sdk";
2
3  let sdk: FiloSDK;
4  const cancelamentos: Array<() => void> = [];
5
6  // Chamada pelo Filo ao carregar o módulo
7  export async function init(filoSdk: FiloSDK): Promise<void> {
8    sdk = filoSdk;
9    // registrar UI, subscrever eventos...
10 }
11
12 // Chamada pelo Filo ao desativar ou recarregar o módulo
13 export async function destroy(): Promise<void> {
14   cancelamentos.forEach((fn) => fn());
15   cancelamentos.length = 0;
16 }
```

### 3.3 O manifesto `module.json`

Todo módulo deve ter um arquivo `module.json` na raiz do projeto. Ele descreve o módulo e declara as permissões de que ele necessita.

`module.json` — manifesto completo anotado

```
1  {
2    "id": "minha-empresa.nome-do-modulo",
3    "name": "Nome Legível do Módulo",
4    "version": "1.0.0",
5    "description": "O que este módulo faz.",
6    "author": "Minha Empresa LTDA",
7    "entryPoint": "dist/index.js",
8    "permissions": [
9      "sdk:data_api",
10     "sdk:ui_api",
11     "sdk:events_api"
12   ],
```

```
13  "extensionPoints": [  
14    {  
15      "type": "sidebar_item",  
16      "slot": "sidebar.main",  
17      "label": "Meu Módulo",  
18      "icon": "ti-puzzle"  
19    }  
20  ]  
21 }
```

| Campo           | Obrigatório | Descrição                                                     |
|-----------------|-------------|---------------------------------------------------------------|
| id              | Sim         | Identificador único: empresa.modulo (minúsculas, sem espaços) |
| name            | Sim         | Nome legível exibido na interface                             |
| version         | Sim         | Versão semântica: MAJOR.MINOR.PATCH                           |
| entryPoint      | Sim         | Caminho para o JS compilado (relativo à raiz)                 |
| permissions     | Sim         | Lista de APIs do SDK que o módulo utiliza                     |
| extensionPoints | Sim         | Pontos da interface onde o módulo se registra                 |
| description     | Não         | Descrição exibida no gerenciador de módulos                   |
| author          | Não         | Nome do desenvolvedor ou empresa                              |

### 3.4 A Filo SDK — visão geral das APIs

A Filo SDK é o conjunto de APIs disponibilizadas pelo Filo para os módulos. Ela é injetada como parâmetro na função `init()` e está disponível através do objeto passado:

| Namespace  | Permissão         | O que oferece                                 |
|------------|-------------------|-----------------------------------------------|
| sdk.data   | sdk:data_api      | Consultas SQL e mutações no banco do Filo     |
| sdk.ui     | sdk:ui_api        | Registro de telas, modais, sidebars e toasts  |
| sdk.events | sdk:events_api    | Publicação e subscrição de eventos do sistema |
| sdk.utils  | sdk:utils_api     | Formatação, armazenamento local e HTTP        |
| sdk.meta   | Sempre disponível | ID do módulo, versão e ID do usuário logado   |



# 04

## A Filo SDK em Detalhes

*Data API, UI API, Events API e Utils API com exemplos práticos.*

### 4.1 Data API — consultando e gravando dados

A **Data API** permite que módulos leiam e escrevam no banco PostgreSQL do Filo. As consultas são sempre parametrizadas — nunca construa SQL por concatenação de strings.

Data API — consultas, mutações e transações

```
1  // query<T> – retorna array de resultados
2  const contatos = await sdk.data.query<{
3    id: string;
4    name: string;
5    email: string | null;
6  }>(
7    "SELECT id, name, email FROM contacts",
8    " WHERE is_active = true ORDER BY name"
9  );
10
11 // queryOne<T> – retorna um registro ou null
12 const contato = await sdk.data.queryOne<{ name: string }>(
13   "SELECT name FROM contacts WHERE id = $1",
14   [contatoId]
15 );
16 if (contato) console.log(contato.name);
17
18 // execute – INSERT, UPDATE, DELETE
19 const res = await sdk.data.execute(
20   "INSERT INTO activities (contact_id, type, title) VALUES ($1, $2, $3)",
21   [contatoId, "note", "Ligação realizada"]
22 );
23 console.log(`${res.rowCount} linha(s) afetada(s)`);
24
25 // transaction – garante atomicidade
```

```
26 await sdk.data.transaction(async (tx) => {
27   await tx.execute("UPDATE opportunities SET stage=$1 WHERE id=$2",
28     ["fechado_ganho", oppId]);
29   await tx.execute("INSERT INTO activities (type,title) VALUES ($1,$2)",
30     ["note", "Venda concluída"]);
31   // Se qualquer linha falhar, ambas são revertidas
32 });
```

- Módulos podem criar suas **próprias tabelas** no banco do Filo usando `sdk.data.execute("CREATE TABLE IF NOT EXISTS...")`. Recomenda-se prefixar o nome da tabela com o ID do módulo: `mod_meumodulo_registros`.

## 4.2 UI API — registrando telas e componentes

A **UI API** permite que módulos registrem elementos visuais dentro do Filo. Todos os componentes são retornados como `HTMLElement`.

UI API — registrando componentes visuais

```
1 // Registrar item na sidebar
2 sdk.ui.registerSidebarItem({
3   slot: "sidebar.main",
4   label: "Meu Módulo",
5   icon: "ti-puzzle",
6   component: () => renderizarTela(),
7 });
8
9 // Registrar aba em contato
10 sdk.ui.registerTab("contact", {
11   label: "Histórico Externo",
12   component: (contatoId: string) => renderizarHistorico(contatoId),
13 });
14
15 // Registrar aba em oportunidade
16 sdk.ui.registerTab("opportunity", {
17   label: "Checklist",
18   component: (oppId: string) => renderizarChecklist(oppId),
19 });
20
21 // Exibir modal
22 await sdk.ui.showModal({
```

```
23   title: "Confirmar ação",
24   content: construirFormulario(),
25   width: 480,
26 });
27
28 // Toasts de notificação
29 sdk.ui.showToast("Salvo com sucesso!", "success");
30 sdk.ui.showToast("Ocorreu um erro.", "error");
31 sdk.ui.showToast("Atenção: verifique.", "info");
```

## Construindo um HTMLElement simples para a tela principal:

Construindo HTMLElement com dados assíncronos

```
1   function renderizarTela(): HTMLElement {
2     const container = document.createElement("div");
3     container.style.cssText = "padding:1.5rem;color:#fff;";
4
5     container.innerHTML = `
6     <h2 style="font-size:15px;font-weight:500;margin-bottom:8px">
7     Meu Módulo
8     </h2>
9     <p style="font-size:13px;color:#9CA3AF">
10    Conteúdo carregando...
11    </p>
12    `;
13
14    // Carregar dados de forma assíncrona
15    carregarDados(container);
16
17    return container;
18  }
19
20  async function carregarDados(container: HTMLElement): Promise<void> {
21    try {
22      const dados = await sdk.data.query("SELECT * FROM contacts LIMIT 10");
23      // atualizar container com os dados...
24    } catch (erro) {
25      sdk.ui.showToast("Erro ao carregar dados.", "error");
26    }
27  }
```



## 4.3 Events API — reagindo a eventos do sistema

A **Events API** implementa o padrão publish/subscribe. Módulos se inscrevem em eventos emitidos pelo Filo e reagem a eles. A função de inscrição retorna uma função de cancelamento — que deve ser chamada no `destroy()`.

Events API — publicar e subscrever eventos

```
1 // Subscrever evento – guardar função de cancelamento
2 const cancelar = sdk.events.on(
3   "opportunity.closed_won",
4   async (payload: { opportunityId: string; value: number }) => {
5     console.log(`Oportunidade ${payload.opportunityId} fechada!`);
6     await processarVenda(payload.opportunityId, payload.value);
7   }
8 );
9
10 // Registrar no array para cancelar no destroy()
11 cancelamentos.push(cancelar);
12
13 // Emitir evento customizado do módulo
14 // Prefixo obrigatório: id-do-modulo.nome_evento
15 sdk.events.emit(
16   "minha-empresa.meu-modulo.processamento_concluido",
17   { status: "ok", timestamp: new Date().toISOString() }
18 );
```

| Evento                    | Quando é emitido                  |
|---------------------------|-----------------------------------|
| contact.created           | Novo contato cadastrado           |
| contact.updated           | Contato atualizado                |
| opportunity.created       | Nova oportunidade criada          |
| opportunity.stage_changed | Oportunidade mudou de etapa       |
| opportunity.closed_won    | Oportunidade marcada como ganha   |
| opportunity.closed_lost   | Oportunidade marcada como perdida |
| proposal.accepted         | Proposta aceita pelo cliente      |
| activity.completed        | Atividade marcada como concluída  |

| Evento    | Quando é emitido             |
|-----------|------------------------------|
| app.ready | Filo terminou de inicializar |

## 4.4 Utils API — utilitários e armazenamento

Utils API — formatação, storage e HTTP

```
1 // Formatação monetária (BRL)
2 sdk.utils.formatCurrency(1234.56); // "R$ 1.234,56"
3
4 // Formatação de data (DD/MM/AAAA)
5 sdk.utils.formatDate(new Date()); // "28/05/2026"
6
7 // Geração de UUID
8 const id = sdk.utils.generateId(); // "a1b2c3d4-e5f6-..."
9
10 // Armazenamento local por módulo (persiste entre sessões)
11 await sdk.utils.storage.set("ultima_sync", new Date().toISOString());
12 const ultima = await sdk.utils.storage.get("ultima_sync");
13 await sdk.utils.storage.delete("ultima_sync");
14
15 // Requisições HTTP externas (proxiadas pelo processo principal)
16 const resposta = await sdk.utils.http.get(
17   "https://api.minha-empresa.com.br/dados",
18   { "Authorization": "Bearer token123" }
19 );
20
21 // POST com corpo JSON
22 const resultado = await sdk.utils.http.post(
23   "https://api.minha-empresa.com.br/registrar",
24   { id: contato.id, nome: contato.nome },
25   { "Content-Type": "application/json" }
26 );
```

# 05

## Criando o Primeiro Módulo

*Do scaffold ao hot reload: passo a passo com o Filo Studio.*

### 5.1 Scaffold com o Filo Studio

O Filo Studio é a IDE embutida no Filo CRM, acessível em **Configurações** → **Studio**. Ele oferece editor de código com IntelliSense, visualização em tempo real e terminal integrado.

- 1 Abra o Filo e acesse **Configurações** → **Studio**.
- 2 Clique em **Novo módulo** na barra superior.
- 3 Informe o nome do módulo (ex: meu-modulo-teste).
- 4 O Studio criará a estrutura completa em `~/filo-modulos/meu-modulo-teste/`.
- 5 O projeto abrirá automaticamente no editor.

■ Alternativamente, você pode desenvolver no VS Code externo. Abra a pasta `~/filo-modulos/seu-modulo/` no VS Code e o Filo Studio detectará automaticamente as mudanças salvas.

### 5.2 Estrutura de arquivos do módulo

Estrutura gerada pelo Filo Studio

```
1  meu-modulo-teste/
2  ■■■ module.json ← Manifesto do módulo
3  ■■■ package.json ← Dependências e scripts
4  ■■■ tsconfig.json ← Configuração TypeScript
5  ■■■ esbuild.config.js ← Build com hot reload
6  ■■■ src/
```

```
7  ■ ■■■ index.ts ← Ponto de entrada (init + destroy)
8  ■ ■■■ views/
9  ■ ■■■ main.view.ts ← Tela principal
10 ■■■ dist/
11 ■ ■■■ index.js ← JS compilado (gerado automaticamente)
12 ■■■ tests/
13 ■ ■■■ index.test.ts ← Testes unitários
14 ■■■ README.md
```

## 5.3 Implementando init() e destroy()

Abra o arquivo `src/index.ts`. O Studio gera este conteúdo inicial:

`src/index.ts` — código inicial gerado pelo Studio

```
1  import type { FiloSDK } from "@w3ti/filo-sdk";
2
3  // Referência ao SDK – preenchida no init()
4  let sdk: FiloSDK;
5
6  // Funções de cancelamento de eventos
7  const cancelamentos: Array<() => void> = [];
8
9  /**
10 * init – chamada pelo Filo ao carregar o módulo.
11 */
12 export async function init(filoSdk: FiloSDK): Promise<void> {
13   sdk = filoSdk;
14
15   sdk.ui.registerSidebarItem({
16     slot: "sidebar.main",
17     label: "Meu Módulo",
18     icon: "ti-puzzle",
19     component: () => renderizarTela(),
20   });
21
22   console.log(`[${sdk.meta.moduleId}] Módulo inicializado.`);
23 }
24
25 /**
26 * destroy – chamada pelo Filo ao desativar o módulo.
27 */
```

```
28 export async function destroy(): Promise<void> {
29   cancelamentos.forEach((fn) => fn());
30   cancelamentos.length = 0;
31   console.log(`[${sdk.meta.moduleId}] Módulo encerrado.`);
32 }
33
34 function renderizarTela(): HTMLElement {
35   const el = document.createElement("div");
36   el.style.padding = "1.25rem";
37   el.innerHTML = `<h2 style="color:#fff">Olá do Meu Módulo!</h2>`;
38   return el;
39 }
```

## 5.4 Registrando um item na sidebar

O tipo de extensão mais comum é o **item de sidebar**. A função `component` é chamada pelo Filo toda vez que o usuário clica no item — ela deve retornar um `HTMLElement` com o conteúdo a ser exibido.

Registrando item na sidebar com conteúdo dinâmico

```
1   sdk.ui.registerSidebarItem({
2     slot: "sidebar.main", // posição na barra lateral
3     label: "Tarefas", // texto exibido
4     icon: "ti-checklist", // ícone Tabler
5     component: () => {
6       const container = document.createElement("div");
7       container.style.cssText = [
8         "padding: 1.25rem;",
9         "display: flex;",
10        "flex-direction: column;",
11        "gap: 0.75rem;",
12      ].join(" ");
13
14      // Título
15      const titulo = document.createElement("h2");
16      titulo.style.cssText = "font-size:15px;font-weight:500;color:#fff;";
17      titulo.textContent = "Minhas Tarefas";
18      container.appendChild(titulo);
19
20      // Carregar lista de tarefas
21      carregarTarefas(container);
22    }
```

```
23   return container;  
24 },  
25 });
```

## 5.5 Compilando e testando com hot reload

No Filo Studio, clique no botão  **Executar**. O Studio irá:

1. Compilar o TypeScript com esbuild em modo desenvolvimento.
  -
2. Carregar o módulo compilado no painel de preview ao lado do editor.
  -
3. Exibir logs do console.log no painel de terminal na base da tela.
  -
4. A cada **Ctrl+S**, recompilar e recarregar automaticamente (**hot reload**).
  -

 Você também pode compilar manualmente pelo terminal do Studio com `npm run build` (produção) ou `npm run dev` (modo watch com hot reload).

# 06

## Módulo Completo: Histórico de Interações

*Um módulo real integrando Data API, UI API e Events API.*

### 6.1 Planejamento e manifesto

Neste capítulo construiremos um módulo completo: o **Histórico de Interações**. Ele adiciona uma aba na tela de contatos do Filo para registrar e visualizar interações personalizadas (reuniões, ligações, e-mails) além das atividades nativas.

O módulo terá as seguintes funcionalidades:

- Aba "Histórico" na tela de cada contato
- Listagem de interações com data, tipo e descrição
- Formulário para registrar nova interação
- Contador de interações exibido na aba
- Notificação automática ao fechar uma oportunidade

module.json — módulo Histórico de Interações

```
1  {
2    "id": "w3ti.historico-interacoes",
3    "name": "Histórico de Interações",
4    "version": "1.0.0",
5    "description": "Registra interações personalizadas por contato.",
6    "author": "W3TI SERVIÇOS DE INFORMÁTICA LTDA",
7    "entryPoint": "dist/index.js",
8    "permissions": [
9      "sdk:data_api",
10     "sdk:ui_api",
11     "sdk:events_api"
12   ],
13   "extensionPoints": [
14     {
15       "type": "contact_tab",
16       "slot": "contacts.tabs",
```

```
17  "label": "Histórico"
18  }
19  ]
20  }
```

## 6.2 Criando tabela própria do módulo

src/database.ts — funções de acesso ao banco

```
1  // src/database.ts
2  import type { FiloSDK } from "@w3ti/filo-sdk";
3
4  export async function inicializarBanco(sdk: FiloSDK): Promise<void> {
5    await sdk.data.execute(`
6      CREATE TABLE IF NOT EXISTS mod_historico_interacoes (
7        id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
8        contact_id uuid NOT NULL,
9        tipo varchar(50) NOT NULL,
10       descricao text NOT NULL,
11       data_hora timestamptz DEFAULT now(),
12       criado_por varchar(200)
13     )
14   `);
15 }
16
17 export async function listarInteracoes(
18   sdk: FiloSDK,
19   contactId: string
20 ): Promise<Interacao[]> {
21   return sdk.data.query<Interacao>(
22     "SELECT * FROM mod_historico_interacoes",
23     " WHERE contact_id = $1 ORDER BY data_hora DESC",
24     [contactId]
25   );
26 }
27
28 export async function registrarInteracao(
29   sdk: FiloSDK,
30   dados: NovaInteracao
31 ): Promise<void> {
32   await sdk.data.execute(
```



```
33  "INSERT INTO mod_historico_interacoes",
34  " (contact_id, tipo, descricao, criado_por) VALUES ($1,$2,$3,$4)",
35  [dados.contactId, dados.tipo, dados.descricao, dados.criadoPor]
36  );
37  }
38
39  export interface Interacao {
40    id: string;
41    contact_id: string;
42    tipo: string;
43    descricao: string;
44    data_hora: string;
45    criado_por: string | null;
46  }
47
48  export interface NovaInteracao {
49    contactId: string;
50    tipo: string;
51    descricao: string;
52    criadoPor: string;
53  }
```

## 6.3 Tela de listagem com busca

src/views/historico.view.ts — tela de listagem

```
1  // src/views/historico.view.ts
2  import type { FiloSDK } from "@w3ti/filo-sdk";
3  import { listarInteracoes, type Interacao } from "../database";
4
5  export function renderizarHistorico(
6    sdk: FiloSDK,
7    contactId: string
8  ): HTMLElement {
9    const container = document.createElement("div");
10   container.style.cssText =
11     "padding:1rem;display:flex;flex-direction:column;gap:.75rem";
12
13   const header = document.createElement("div");
14   header.style.cssText =
15     "display:flex;align-items:center;justify-content:space-between";
```

```
16 header.innerHTML = `
17 <h3 style="font-size:14px;font-weight:500;color:#fff">Interações</h3>
18 <button id="btn-nova"
19 style="background:#7F77DD;color:#fff;border:none;border-radius:6px;
20 padding:6px 14px;font-size:12px;cursor:pointer">
21 + Nova interação
22 </button>
23 `;
24 container.appendChild(header);
25
26 const lista = document.createElement("div");
27 lista.id = "lista-interacoes";
28 lista.style.cssText = "display:flex;flex-direction:column;gap:8px";
29 lista.textContent = "Carregando...";
30 container.appendChild(lista);
31
32 // Carregar interações
33 listarInteracoes(sdk, contactId).then((interacoes) => {
34 lista.innerHTML = "";
35 if (interacoes.length === 0) {
36 lista.innerHTML =
37 '<p style="color:#6B7280;font-size:13px">Nenhuma interação registrada.</p>';
38 return;
39 }
40 interacoes.forEach((i) => lista.appendChild(criarCard(i, sdk)));
41 }).catch(() =>
42 sdk.ui.showToast("Erro ao carregar histórico.", "error"));
43
44 // Botão nova interação
45 header.querySelector("#btn-nova")!.addEventListener("click", () => {
46 abrirFormulario(sdk, contactId, lista);
47 });
48
49 return container;
50 }
51
52 function criarCard(i: Interacao, sdk: FiloSDK): HTMLElement {
53 const card = document.createElement("div");
54 card.style.cssText = [
55 "background:#1A1D27;border:0.5px solid #2A2D3A;",
```

```
56 "border-radius:8px;padding:12px;",
57 ].join("");
58 card.innerHTML = `
59 <div style="display:flex;align-items:center;justify-content:space-between;
60 margin-bottom:6px">
61   <span
62     style="font-size:11px;font-weight:500;color:#7F77DD;text-transform:uppercase">
63     ${i.tipo}
64   </span>
65   <span style="font-size:11px;color:#6B7280">
66     ${sdk.utils.formatDate(i.data_hora)}
67   </span>
68 </div>
69 <p style="font-size:13px;color:#9CA3AF">${i.descricao}</p>
70 `;
71 return card;
72 }
```

## 6.4 Formulário de nova interação

Formulário modal para nova interação

```
1  async function abrirFormulario(
2    sdk: FiloSDK,
3    contactId: string,
4    lista: HTMLInputElement
5  ): Promise<void> {
6    const form = document.createElement("div");
7    form.style.cssText =
8      "display:flex;flex-direction:column;gap:12px;padding:4px 0";
9    form.innerHTML = `
10   <div>
11     <label style="font-size:12px;color:#9CA3AF">
12       Tipo de interação
13     </label>
14     <select id="tipo"
15       style="width:100%;margin-top:4px;padding:8px;background:#1A1D27;
16       color:#fff;border:0.5px solid #2A2D3A;border-radius:6px">
17       <option value="reuniao">Reunião</option>
18       <option value="ligacao">Ligação</option>
19       <option value="email">E-mail</option>
20       <option value="visita">Visita presencial</option>
```

```
21 <option value="outro">Outro</option>
22 </select>
23 </div>
24 <div>
25 <label style="font-size:12px;color:#9CA3AF">Descrição</label>
26 <textarea id="descricao" rows="4"
27 style="width:100%;margin-top:4px;padding:8px;background:#1A1D27;
28 color:#fff;border:0.5px solid #2A2D3A;border-radius:6px;
29 resize:vertical;font-family:inherit;font-size:13px"
30 placeholder="Descreva o que foi discutido..."></textarea>
31 </div>
32 <button id="btn-salvar"
33 style="background:#7F77DD;color:#fff;border:none;border-radius:6px;
34 padding:8px 16px;font-size:13px;
35 font-weight:500;cursor:pointer">
36 Salvar interação
37 </button>
38 `;
39
40 await sdk.ui.showModal({
41 title: "Nova Interação", content: form, width: 480
42 });
43
44 // Capturar dados após o modal fechar
45 const tipo = (form.querySelector("#tipo") as HTMLSelectElement).value;
46 const descricao = (
47 form.querySelector("#descricao") as HTMLTextAreaElement
48 ).value;
49
50 if (!descricao.trim()) {
51 sdk.ui.showToast("A descrição é obrigatória.", "error");
52 return;
53 }
54
55 try {
56 await registrarInteracao(sdk, {
57 contactId, tipo, descricao,
58 criadoPor: sdk.meta.userId,
59 });
60 sdk.ui.showToast("Interação registrada!", "success");
```

```
61
62 // Recarregar lista
63 const interacoes = await listarInteracoes(sdk, contactId);
64 lista.innerHTML = "";
65 interacoes.forEach((i) => lista.appendChild(criarCard(i, sdk)));
66 } catch {
67   sdk.ui.showToast("Erro ao salvar interação.", "error");
68 }
69 }
```

## 6.5 Publicando o módulo

Com o módulo testado e funcionando, clique em **Publicar** no Filo Studio. O processo gera um arquivo `.filomod` que pode ser:

- Instalado manualmente em outros Filos via **Configurações** → **Módulos** → **Instalar**
- Distribuído diretamente aos clientes por e-mail ou download
- Publicado no Marketplace Filo (requer conta W3TI de desenvolvedor)

# 07

## Boas Práticas e Distribuição

*Segurança, erros, testes e versionamento.*

### 7.1 Segurança e permissões mínimas

Declare apenas as permissões estritamente necessárias no `module.json`. Usuários podem visualizar as permissões de um módulo antes de instalá-lo — módulos com permissões excessivas geram desconfiança.

| Permissão                   | Quando usar                                          |
|-----------------------------|------------------------------------------------------|
| <code>sdk:data_api</code>   | Apenas quando o módulo precisa ler ou escrever dados |
| <code>sdk:ui_api</code>     | Sempre que o módulo exibir qualquer elemento visual  |
| <code>sdk:events_api</code> | Apenas quando o módulo reage ou emite eventos        |
| <code>sdk:utils_api</code>  | Para formatação, storage local ou requisições HTTP   |

### 7.2 Tratamento de erros

Padrões de tratamento de erro

```
1 // ■ Incorreto – erro não tratado
2 const dados = await sdk.data.query("SELECT * FROM contacts");
3
4 // ✓ Correto – sempre use try/catch em código assíncrono
5 try {
6   const dados = await sdk.data.query("SELECT * FROM contacts");
7   renderizarLista(dados);
8 } catch (erro) {
9   sdk.ui.showToast("Não foi possível carregar os dados.", "error");
10  console.error(`[${sdk.meta.moduleId}] Erro:`, erro);
11 }
12
13 // Utilitário reutilizável para operações seguras
```

```
14  async function tentarExecutar<T>(  
15    sdk: FiloSDK,  
16    operacao: () => Promise<T>,  
17    mensagemErro: string  
18  ): Promise<T | null> {  
19    try {  
20      return await operacao();  
21    } catch (erro) {  
22      sdk.ui.showToast(mensagemErro, "error");  
23      console.error(`[módulo]`, mensagemErro, erro);  
24      return null;  
25    }  
26  }
```

## 7.3 Testes com node:test

Exemplo de teste com node:test

```
1  // tests/calculos.test.ts  
2  import { describe, it } from "node:test";  
3  import assert from "node:assert/strict";  
4  import { calcularTotal } from "../src/utils";  
5  
6  describe("calcularTotal", () => {  
7    it("retorna soma correta", () => {  
8      const itens = [{ valor: 100, qtd: 2 }, { valor: 50, qtd: 1 }];  
9      assert.equal(calcularTotal(itens), 250);  
10   });  
11  
12   it("retorna zero para lista vazia", () => {  
13     assert.equal(calcularTotal([]), 0);  
14   });  
15  
16   it("lança erro para valor negativo", () => {  
17     assert.throws(() => calcularTotal([{ valor: -10, qtd: 1 }]));  
18   });  
19   });
```

Execute os testes pelo terminal do Studio ou externamente:

Executando testes

```
1  npm test
```

```
2
3 # Saída esperada:
4 # ✓ calcularTotal > retorna soma correta
5 # ✓ calcularTotal > retorna zero para lista vazia
6 # ✓ calcularTotal > lança erro para valor negativo
7 # ■ tests 3, pass 3, fail 0
```

## 7.4 Versionamento e changelog

Utilize versionamento semântico (**SemVer**) no campo `version` do `module.json`:

| Tipo de mudança                  | Versão         | Exemplo       |
|----------------------------------|----------------|---------------|
| Correção de bug                  | PATCH: X.Y.Z+1 | 1.0.0 → 1.0.1 |
| Nova funcionalidade (compatível) | MINOR: X.Y+1.0 | 1.0.0 → 1.1.0 |
| Mudança incompatível             | MAJOR: X+1.0.0 | 1.0.0 → 2.0.0 |

Mantenha um arquivo `CHANGELOG.md` na raiz do módulo:

`CHANGELOG.md` — exemplo de versionamento

```
1 # Changelog
2
3 ## [1.1.0] - 2026-06-10
4 ### Adicionado
5 - Filtro por tipo de interação na listagem
6 - Exportação do histórico em CSV
7
8 ### Corrigido
9 - Erro ao carregar histórico com mais de 100 registros
10
11 ## [1.0.0] - 2026-05-28
12 ### Lançamento inicial
13 - Aba de histórico na tela de contatos
14 - Formulário de nova interação
```



## Apêndice A

# Referência da Filo SDK

filo-sdk.d.ts — interface completa

```
1  interface FiloSDK {
2    data: {
3      query<T>(sql: string, ...params: unknown[]): Promise<T[]>;
4      queryOne<T>(sql: string, ...params: unknown[]): Promise<T | null>;
5      execute(sql: string, ...params: unknown[]): Promise<{ rowCount: number }>;
6      transaction<T>(fn: (tx: DataAPI) => Promise<T>): Promise<T>;
7    };
8    ui: {
9      registerSidebarItem(item: {
10        slot: string; label: string; icon: string;
11        component: () => HTMLElement;
12      }): void;
13      registerTab(target: "contact" | "opportunity", tab: {
14        label: string;
15        component: (id: string) => HTMLElement;
16      }): void;
17      registerDashboardWidget(widget: {
18        slot: string; title: string; width: "full" | "half";
19        component: () => HTMLElement;
20      }): void;
21      showModal(opts: { title: string; content: HTMLElement; width?: number }): Promise<void>;
22      showToast(message: string, type?: "success" | "error" | "info"): void;
23      navigate(route: string): void;
24    };
25    events: {
26      on(event: string, handler: (payload: unknown) => void): () => void;
27      off(event: string, handler: (payload: unknown) => void): void;
28      emit(event: string, payload: unknown): void;
29    };
30    utils: {
31      formatCurrency(value: number): string;
32      formatDate(date: string | Date): string;
33      generateId(): string;
34    };
35    storage: {
```

```
35  get(key: string): Promise<string | null>;
36  set(key: string, value: string): Promise<void>;
37  delete(key: string): Promise<void>;
38  };
39  http: {
40    get(url: string, headers?: Record<string, string>): Promise<unknown>;
41    post(url: string, body: unknown, headers?: Record<string, string>):
42    Promise<unknown>;
43  };
44  meta: {
45    moduleId: string;
46    version: string;
47    userId: string;
48  };
49  }
```

## Apêndice B

# Eventos nativos do Filo CRM

| Evento                    | Payload                   | Quando é emitido             |
|---------------------------|---------------------------|------------------------------|
| contact.created           | { contactId }             | Novo contato cadastrado      |
| contact.updated           | { contactId }             | Dados do contato atualizados |
| contact.deleted           | { contactId }             | Contato removido             |
| opportunity.created       | { opportunityId }         | Nova oportunidade criada     |
| opportunity.stage_changed | { opportunityId, stage }  | Etapa alterada               |
| opportunity.closed_won    | { opportunityId, value }  | Marcada como ganha           |
| opportunity.closed_lost   | { opportunityId, reason } | Marcada como perdida         |
| proposal.created          | { proposalId }            | Nova proposta criada         |
| proposal.sent             | { proposalId }            | Proposta enviada             |
| proposal.accepted         | { proposalId, value }     | Proposta aceita              |
| proposal.rejected         | { proposalId }            | Proposta rejeitada           |
| activity.created          | { activityId, type }      | Nova atividade criada        |
| activity.completed        | { activityId }            | Atividade concluída          |
| app.ready                 | { }                       | Filo terminou de inicializar |
| app.before_shutdown       | { }                       | Filo será fechado            |

Apêndice C

# Referência do module.json

| Campo           | Tipo     | Obrig. | Descrição                                        |
|-----------------|----------|--------|--------------------------------------------------|
| id              | string   | Sim    | Identificador único: empresa.modulo (minúsculas) |
| name            | string   | Sim    | Nome legível exibido na interface                |
| version         | string   | Sim    | Versão semântica: MAJOR.MINOR.PATCH              |
| entryPoint      | string   | Sim    | Caminho para o JS compilado                      |
| permissions     | string[] | Sim    | Lista de permissões SDK necessárias              |
| extensionPoints | array    | Sim    | Pontos da UI onde o módulo se registra           |
| description     | string   | Não    | Descrição curta do módulo                        |
| author          | string   | Não    | Nome do desenvolvedor ou empresa                 |
| icon            | string   | Não    | SVG em base64 para o marketplace                 |

Valores válidos para o campo type em extensionPoints:

| type             | slot sugerido      | Descrição                          |
|------------------|--------------------|------------------------------------|
| sidebar_item     | sidebar.main       | Item na barra lateral principal    |
| contact_tab      | contacts.tabs      | Aba na tela de detalhes de contato |
| opportunity_tab  | opportunities.tabs | Aba na tela de oportunidade        |
| dashboard_widget | dashboard.widgets  | Widget no dashboard                |
| settings_page    | settings.pages     | Página nas configurações           |